

Proyecto “Capture The Flag”

Integrantes:

Emely Batres - 22004903

Victor Arias – 15001915

Lenguaje: Python

Tecnología de conexión: Sockets

Librerías Gráficas: Pygame

Documentación

Repositorio en GitHub: <https://github.com/emy-707/capture-the-flag-project-cc8>

Versiones

Commit 1: [Estructura inicial de carpetas y README](#)

Se empezó el proyecto armando una estructura del proyecto (para división de trabajo en equipo). Para esto, se pidió una sugerencia a la IA generativa “Gemini”.

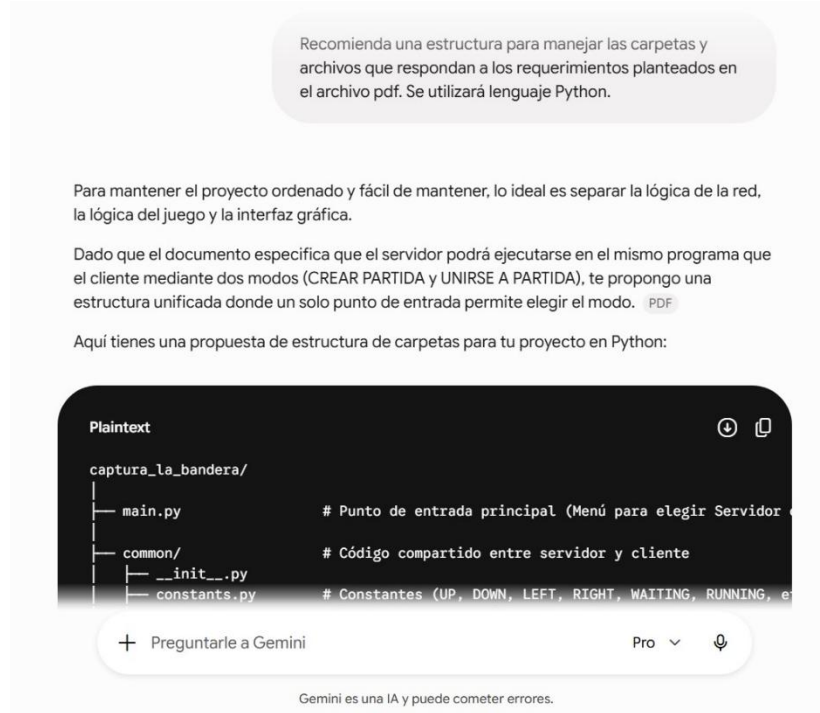
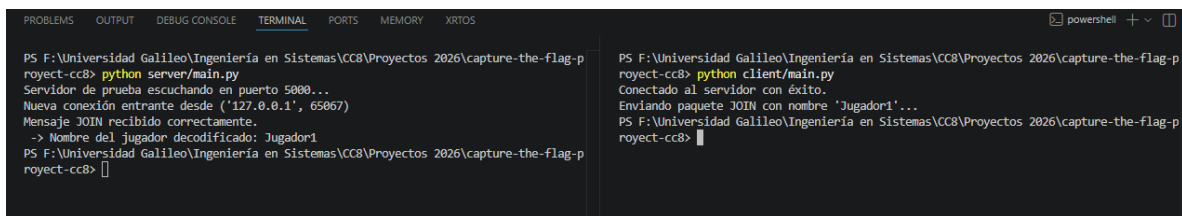


Ilustración 1 Prompt inicial

Luego empezó la comunicación grupal. Inicialmente se comenzó la implementación del proyecto siguiendo un protocolo en el cual se enviaba la información utilizando JSON, sin embargo, se vio con el grupo de clase que esto no era viable por lo que se hizo una reingeniería del protocolo y se decidió utilizar bytes para indicar todos los posibles estados del juego y las acciones de los jugadores.

Commit 2: Implementación de TCP/UDP entre cliente y servidor

Lo principal fue comenzar implementando el protocolo, la lectura de los bytes a través del socket, para ello necesitamos tanto funciones de recepción y transferencia (recv_exact, receive_tcp_message, send_tcp_message) como de conversión (pack_str, unpack_str, pack_coord, unpack_coord) y se probó la primera comunicación entre el cliente y servidor.



```
PS F:\Universidad Galileo\Ingeniería en Sistemas\CC8\Proyectos 2026\capture-the-flag-p
royect-cc8> python server/main.py
Servidor de prueba escuchando en puerto 5000...
Nueva conexión entrante desde ('127.0.0.1', 65067)
Mensaje JOIN recibido correctamente.
-> Nombre del jugador decodificado: Jugador1
PS F:\Universidad Galileo\Ingeniería en Sistemas\CC8\Proyectos 2026\capture-the-flag-p
royect-cc8>

PS F:\Universidad Galileo\Ingeniería en Sistemas\CC8\Proyectos 2026\capture-the-flag-p
royect-cc8> python client/main.py
Conectado al servidor con éxito.
Enviando paquete JOIN con nombre 'Jugador1'...
PS F:\Universidad Galileo\Ingeniería en Sistemas\CC8\Proyectos 2026\capture-the-flag-p
royect-cc8>
```

Ilustración 2 - Resultados en terminal

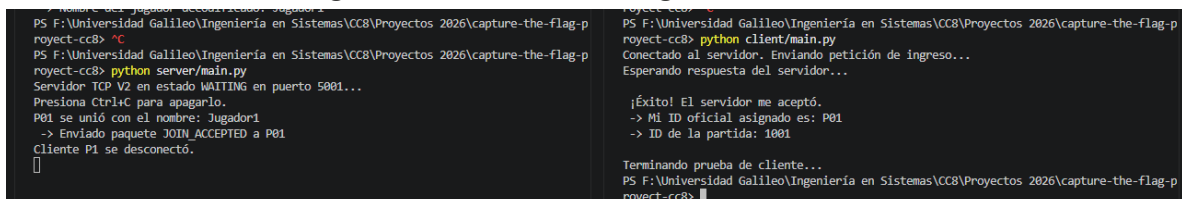
Commit 3: Se define bucle no bloqueante y handshake bidireccional

Posteriormente se estableció un handshake bidireccional que ocurre de la siguiente forma:

1. El cliente envía un paquete de solicitud de conexión “JOIN” con los bytes 0x10
2. El servidor recibe el paquete y verifica que cumpla con tener el mismo protocolo (versión 3)
3. El servidor responde indicando si se aceptó la conexión “JOIN_ACEPTED” con los bytes 0x20 o “JOIN_REJECTED” con los bytes 0x21

También se estableció un bucle “no bloqueante”, es decir que el servidor no se detendrá a esperar la respuesta de uno de los clientes. Esto se realiza con la función: client_socket.setblocking(False)

Una vez el cliente consigue conectarse se le asigna un ID.



```
PS F:\Universidad Galileo\Ingeniería en Sistemas\CC8\Proyectos 2026\capture-the-flag-p
royect-cc8> ^C
PS F:\Universidad Galileo\Ingeniería en Sistemas\CC8\Proyectos 2026\capture-the-flag-p
royect-cc8> python server/main.py
Servidor TCP V2 en estado WAITING en puerto 5001...
Presiona Ctrl+C para apagarlo.
P01 se unió con el nombre: Jugador1
-> Enviado paquete JOIN_ACEPTED a P01
Cliente P1 se desconectó.

PS F:\Universidad Galileo\Ingeniería en Sistemas\CC8\Proyectos 2026\capture-the-flag-p
royect-cc8> python client/main.py
Conectado al servidor. Enviando petición de ingreso...
Esperando respuesta del servidor...

¡Éxito! El servidor me aceptó.
-> MI ID oficial asignado es: P01
-> ID de la partida: 1001

Terminando prueba de cliente...
PS F:\Universidad Galileo\Ingeniería en Sistemas\CC8\Proyectos 2026\capture-the-flag-p
royect-cc8>
```

Ilustración 3 - Resultados en terminal

Commit 4: Implementación de broadcast y LOBBY STATE

Luego se procedió a implementar el broadcast, esto se realiza del lado del servidor el cual procederá a recibir conexiones desde la IP 0.0.0.0, esto significa que aceptará paquetes de cualquier miembro de la red.

Del lado del cliente se realiza una consulta a la IP 255.255.255.255 la cual el router entiende como un broadcast, es decir, a absolutamente todos los dispositivos conectados en la red, y envía un paquete “DISCOVER_REQUEST” con los bytes 0x01 en el puerto 5001 (UDP).

Como ya teníamos al servidor escuchando este último captura ese paquete y responde a ese único cliente con otro paquete “DISCOVER_RESPONSE” con los bytes 0x02 y con los datos de la partida:

- Nombre del servidor
- Cantidad de jugadores en la sala
- Puerto para la conexión TCP

Una vez el cliente recibe el paquete ya se le preguntará al usuario de todos los servidores que se encontraron, ¿a cuál se desea conectar? y con dicha elección ya se realiza la conexión vía TCP



```

- Partida TCP en puerto 5000
- Descubrimiento UDP en puerto 5001
Radar UDP: Respondí a solicitud de 26.192.234.52
Radar UDP: Respondí a solicitud de 26.192.234.52
Radar UDP: Respondí a solicitud de 26.192.234.52
Radar UDP: Respondí a solicitud de 26.192.234.52

--- ACTUALIZACIÓN DE SALA (1 JUGADORES) ---
-> P03: Jugador

--- ACTUALIZACIÓN DE SALA (2 JUGADORES) ---
-> P03: Jugador
-> P04: Jugador

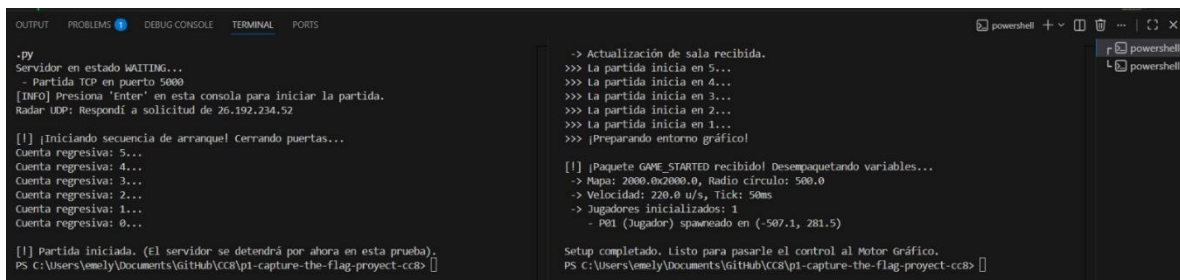
jÉxito! Mi ID oficial es: P04

--- ACTUALIZACIÓN DE SALA (2 JUGADORES) ---
-> P03: Jugador
-> P04: Jugador

```

Commit 5: Se suben cambios

Posteriormente se agregó un countdown para el inicio de la partida, esta acción la realiza el servidor manualmente al recibir un input del usuario.



```

.py
Servidor en estado WAITING...
- Partida TCP en puerto 5000
[INFO] Presiona 'Enter' en esta consola para iniciar la partida.
Radar UDP: Respondí a solicitud de 26.192.234.52

[!] ¡Iniciando secuencia de arranque! Cerrando puertas...
Cuenta regresiva: 5...
Cuenta regresiva: 4...
Cuenta regresiva: 3...
Cuenta regresiva: 2...
Cuenta regresiva: 1...
Cuenta regresiva: 0...

[!] Partida iniciada. (El servidor se detendrá por ahora en esta prueba).
PS C:\Users\emely\Documents\GitHub\CC8\p1-capture-the-flag-project-cc8>

-> Actualización de sala recibida.
>>> La partida inicia en 5...
>>> La partida inicia en 4...
>>> La partida inicia en 3...
>>> La partida inicia en 2...
>>> La partida inicia en 1...
>>> ¡Preparando entorno gráfico!

[!] ¡Paquete GAME_STARTED recibido! Desempaquetando variables...
-> Mapa: 2000.0x2000.0, Radio círculo: 500.0
-> Velocidad: 220.0 u/s, Tick: 50ms
-> Jugadores inicializados: 1
  - P01 (Jugador) spawnado en (-507.1, 281.5)

Setup completado. Listo para pasarle el control al Motor Gráfico.
PS C:\Users\emely\Documents\GitHub\CC8\p1-capture-the-flag-project-cc8>

```

Commit 6: Motor gráfico

Ya teniendo todo esto implementado se procedió a integrar la parte gráfica utilizando pygame, se pidió ayuda a Gemini para dicha implementación. Gemini sugirió crear un loop infinito en el cual se vayan manejando la lectura de los inputs del jugador y el dibujado constante del mapa y los jugadores.

Del lado del servidor se establecieron los estados para diferenciar si el juego está en la sala de espera o si ya está corriendo, y si está corriendo deberá ir actualizando al resto de jugadores la posición del resto.

Commit 7: Implementación de lógica final de juego

asdfa

main.py

Ayúdame a implementar la parte gráfica utilizando la librería pygame, te comparto el código que llevo hasta el momento, el juego será de capturar la bandera, todos los jugadores iniciarán afuera de un círculo y el objetivo es que corran

¡Aquí es donde la magia ocurre y el proyecto cobra vida visualmente!

Para integrar Pygame sin que tu pantalla se congele, la regla de oro es que el cliente también debe usar un socket no bloqueante. Si Pygame se queda esperando a que pasen los 200 milisegundos del servidor para recibir el siguiente mensaje, tu juego correrá a 5 cuadros por segundo (FPS) y los controles se sentirán espantosos. [PDF](#)

Al hacer el socket no bloqueante, permitimos que Pygame gire a 60 FPS fluidos para leer tu teclado al instante, mientras que en segundo plano "atrapa" los estados del servidor cada vez que llegan.

Aquí tienes la estructura base para transformar tu `client/main.py` en un motor gráfico interactivo:

Python



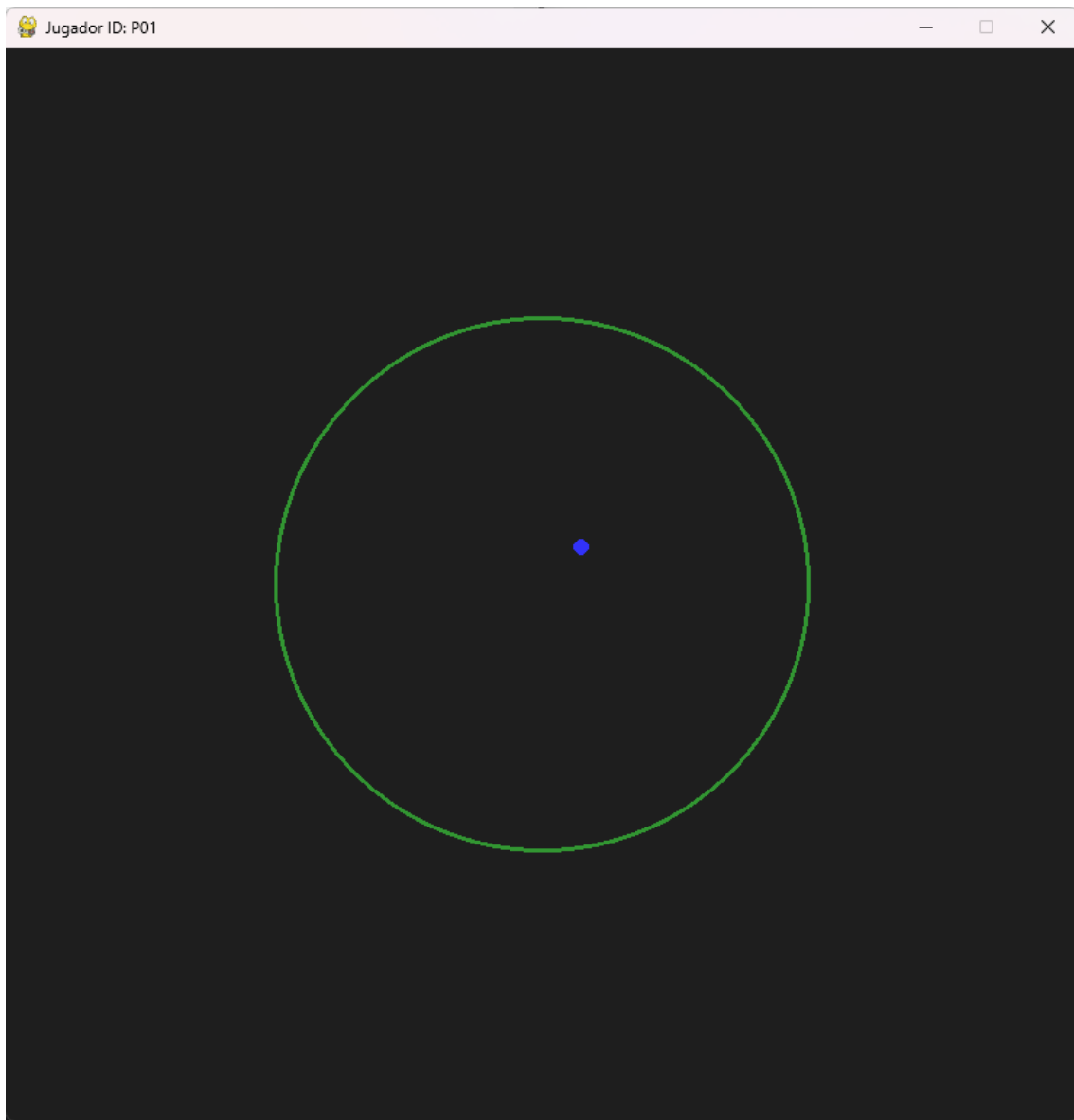
```
import socket
import sys
import os
import pygame

# Ajusta tu ruta según tu estructura
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from common.protocol import send_message, receive_messages

HOST = '127.0.0.1'
PORT = 5000

# Configuraciones visuales del tablero
CELL_SIZE = 30
ROWS, COLS = 20, 20 # Según el parámetro sugerido por el RFC
WIDTH, HEIGHT = COLS * CELL_SIZE, ROWS * CELL_SIZE

# Colores básicos
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
BLUE = (50, 150, 255)
RED = (255, 50, 50)
GREEN = (50, 255, 50)
```



Ya teniendo a los jugadores en el mapa y capturando sus movimientos, por último se implementó la lógica del juego, se agrega la bandera, las acciones de tomar/robar, el cálculo para saber si un portador de la bandera sale del círculo y así dar por finalizada la partida.

¡JUGADOR GANÓ LA PARTIDA!

Jugador

